

Universidade Federal Rural de Pernambuco

Departamento de Computação

Disciplina: Programação Orientada a Objetos

Sistema Web de E-commerce com Arquitetura Multi-seller

Plataforma de E-commerce Multi-vendedor

Equipe:

Joaquim Germano Felix

Mateus Correia Dias

Miguel Mochizuki Silva

Gabriel Bringel Gonçalves

Professor: Gabriel Belarmino

Código-fonte:

<https://github.com/MiguelMochizukiDev/ecommerce>

João Pessoa, 2026

1. Introdução

O projeto consiste no desenvolvimento de uma plataforma de e-commerce multi-vendedor, onde um mesmo usuário pode assumir os papéis de comprador, vendedor ou ambos. A ideia surgiu da percepção de que marketplaces reais impõem desafios arquiteturais que vão além de um simples CRUD: um único carrinho pode conter produtos de lojas distintas, o que exige regras específicas para dividir pedidos, processar pagamentos por vendedor e manter histórico de status de forma independente para cada parte envolvida.

Do ponto de vista técnico, o sistema foi construído com backend em Java 21 usando Spring Boot e frontend em React com TypeScript. A comunicação entre as camadas acontece por meio de uma API REST protegida com autenticação JWT e controle de acesso baseado em papéis (*BUYER*, *SELLER*, *ADMIN*). Essa divisão nos permitiu trabalhar de forma paralela, entender melhor o contrato entre cliente e servidor, e aplicar na prática conceitos de arquitetura em camadas.

A motivação acadêmica foi usar o projeto como laboratório real de orientação a objetos. Não queríamos somente criar classes que funcionassem, mas sim organizar o código de forma que as decisões de modelagem fizessem sentido dentro do domínio do problema. Isso guiou desde o nome das classes até a distribuição de responsabilidades entre serviços, repositórios e controladores.

2. Modelagem do Problema

2.1 Visão geral e domínios

Antes de escrever qualquer linha de código, produzimos diagramas UML para mapear as entidades e seus relacionamentos. A modelagem foi dividida em subdomínios, o que facilitou tanto o entendimento do problema quanto a distribuição de trabalho na equipe:

- **Usuário e vendedor:** *User*, *SellerProfile*, *Role*, *PaymentMethod*
- **Catálogo de produtos:** *Category*, *Product*, *ProductStatus*
- **Carrinho:** *Cart*, *CartItem*
- **Pedidos:** *Order*, *SubOrder*, *SubOrderItem*, *SubOrderStatus*, *SubOrderStatusHistory*
- **Avaliações:** *Review*

A relação mais crítica do sistema é entre *Order* e *SubOrder*: quando um comprador finaliza um pedido com produtos de três vendedores diferentes, o sistema cria automaticamente

três subpedidos, cada um com seu próprio ciclo de vida e histórico de status. Isso reflete uma composição forte no domínio.

2.2 Aspectos de orientação a objetos

Encapsulamento. As regras de domínio ficam nos próprios objetos. A classe `Product`, por exemplo, possui o método `updateStock(int quantity)`, que atualiza a quantidade disponível e ajusta automaticamente o status para `ACTIVE` ou `OUT_OF_STOCK`. Sem esse encapsulamento, essa lógica ficaria duplicada em vários pontos do código.

Abstração. Cada classe representa um conceito do negócio com atributos essenciais, sem expor detalhes de infraestrutura. DTOs formam a fronteira entre a API e o domínio, de modo que entidades JPA nunca chegam diretamente ao frontend.

Composição e agregação. `Order` e `SubOrder` formam uma composição: subpedidos não existem fora de um pedido. `Cart` e `CartItem` seguem lógica semelhante de agregação.

Coesão e responsabilidade única. Cada camada tem um propósito claro: entidade (estado e comportamento de negócio), repositório (acesso a dados), serviço (orquestração de regras), controlador (interface HTTP) e DTO (contrato de entrada/saída). Essa separação evitou classes infladas e tornou o código mais fácil de testar.

Enumerações orientadas ao domínio. Tipos como `ProductStatus`, `SubOrderStatus`, `Role` e `PaymentMethod` substituem cadeias de texto espalhadas pelo código, garantindo segurança de tipos e comportamento condicional claro.

2.3 Convenções adotadas

Para manter consistência entre os integrantes, definimos algumas convenções logo no início:

- Estrutura de pacotes por domínio (`domain/cart`, `domain/order` etc.)
- Endpoints REST nomeados por recurso (`/api/products`, `/api/orders` etc.)
- Validação declarativa com Bean Validation nas entradas da API
- Tratamento centralizado de exceções com respostas padronizadas
- Nomenclatura consistente: `ProductService`, `ProductRepository`, `ProductController`

2.4 Diagramas UML

A documentação UML completa está disponível no diretório `docs/` do repositório, organizada em arquivos PlantUML por domínio

- `overview.puml`

- `action.puml`
- `user-seller-domain.puml`
- `product-catalog-domain.puml`
- `cart-shopping-domain.puml`
- `order-domain.puml`
- `review-domain.puml` `entity-relationship.puml`

Note-se que, além das classes UML, esboçou-se as entidades e relacionamentos do projeto.

3. Ferramentas Utilizadas

IDE e ambiente. Utilizamos o Visual Studio Code com extensões para Java, Spring Boot, TypeScript e visualização de diagramas PlantUML. O gerenciamento de build do backend foi feito com Maven (incluindo wrapper `mvnw`).

Linguagens. Backend em Java 21, frontend em TypeScript com React, e documentação em Markdown e PlantUML.

Backend. O núcleo do backend é o **Spring Boot 3.5.11**, com os seguintes módulos: Spring Web (API REST), Spring Data JPA com Hibernate (persistência), Spring Security (autenticação/autorização), JWT via `io.jsonwebtoken` (jjwt 0.12.6), Bean Validation, MySQL Connector/J e Lombok para redução de boilerplate.

Frontend. React 19 com React Router DOM para navegação, Axios para requisições HTTP, Vite como bundler, Tailwind CSS para estilização e ESLint para qualidade de código.

Estrutura de pacotes — backend:

- `config/` — segurança, CORS e componentes transversais
- `domain/` — subdomínios: `cart`, `category`, `order`, `product`, `review`, `seller`, `user`
- `infra/` — filtros de segurança e componentes de infraestrutura

Estrutura de módulos — frontend:

- `components/` — componentes reutilizáveis
- `pages/` — páginas da aplicação
- `contexts/` — estados globais com Context API

- `services/` — integração com a API REST
- `layouts/` — estrutura de layout

4. Resultados e Considerações Finais

4.1 Resultados alcançados

O sistema chegou ao final do projeto com um conjunto funcional de funcionalidades: autenticação e autorização com múltiplos papéis, fluxo completo de cadastro de vendedor e produtos, carrinho operacional com validações, criação de pedidos com divisão automática em subpedidos por vendedor, suporte a métodos de pagamento configurados por vendedor, e um sistema de avaliações vinculado ao ciclo de compra.

Mais do que as funcionalidades em si, o ganho que mais valorizamos foi estrutural: o código cresceu de forma organizada ao longo do semestre, sem acumular débito técnico excessivo. Quando precisávamos alterar algo, sabíamos onde mexer.

4.2 Dificuldades encontradas

O maior desafio técnico foi modelar corretamente o relacionamento entre `Order` e `SubOrder`. Levou algumas iterações até entender como representar isso de forma que a persistência no banco de dados e as regras de negócio se complementassem sem gambiarras. Outro ponto difícil foi definir até onde vai a responsabilidade de cada camada — em vários momentos a tentação de colocar lógica de negócio no controlador era grande, mas resistimos.

No âmbito da equipe, padronizar nomenclatura e convenções de código entre quatro pessoas exigiu acordos explícitos no começo do projeto. Não fizemos isso cedo o suficiente, e pagamos um preço pequeno de refatoração no meio do caminho.

4.3 Reflexão sobre aprendizagem

A experiência mudou a forma como pensamos em orientação a objetos. Antes do projeto, POO soava como “criar classes com getters e setters”. Na prática, percebemos que o paradigma é sobre modelar o domínio do problema de forma fiel, distribuir responsabilidades de modo coeso e proteger as regras do negócio dentro dos próprios objetos. A decisão de onde colocar um método pode parecer trivial, mas afeta diretamente a qualidade do código semanas depois.

Java como linguagem se mostrou verboso em comparação com o que conhecíamos, mas a verbosidade tem um lado positivo: o código tende a ser explícito, o que ajuda na leitura

e na colaboração. Spring Boot abstraiu muito da infraestrutura e nos deixou focar no domínio, o que foi pedagogicamente interessante.

4.4 Feedback para a disciplina

De forma geral, a proposta do projeto foi bem-vinda porque aproximou a teoria da realidade de mercado. Algumas sugestões:

1. Apresentar frameworks Java para desenvolvimento full-stack, ou esboçar como integrar Java com outros stacks para melhor aprendizagem (Ex.: React, Next etc.)
2. Incentivar testes automatizados como critério de qualidade seria um passo natural para consolidar boas práticas além da modelagem.